AHU fault detection and diagnosis

A personal machine learning project

Sergio Serrano

November 2026

This personal project finds and diagnoses faults in air handling units (AHUs) from 5-minute sensor data of 40 machines in three buildings. It uses a sample dataset, and the scenario in Section 4 is an example. Code and history are on GitHub: https://github.com/serrasm19/ahu-fault-detection.

At a glance

- **Data.** Each building had its own data problem: one office week stamped 8 hours early, a mall day saved twice, an 8-day hospital outage, fan power in watts, CO2 glitches and two frozen sensors. I found, proved and fixed each one, and they become automatic checks in production. I also found a stuck damper that has no work order.
- **Model.** A LightGBM model decides *if* a machine has a fault; physics rules decide *which* one. In validation it found all 29 faults, 93% with the right type, about 10 hours after they started, with about one false alarm per machine every 20 months. It raises 14 alarms in July and August.
- **Production.** Azure and Databricks, an edge agent that only sends data out over HTTPS, hourly inference, a new model only replaces the current one if it is better on the same validation, and alerts designed against alarm fatigue.
- **Communication.** In the example scenario: agree with the facilities manager, check the 9 alerts this week, send only high-confidence alerts, and connect the two new buildings next month without alerts until the pilot shows that 7 of 10 checked alerts are real.

What is in this project

Where	What
`report.pdf`	This report
`predictions.csv`	Fault type for every row of the test period (format checked by `predict.py`)
`code/README.md`	Start here: setup, how to rebuild `predictions.csv` and this report
`code/notebooks/`	`01-04`: data exploration and quality; `05_modelling`: the model
`code/src/ahu_fdd/`	Python package used by the notebooks: cleaning, features, rules, model, scoring, `predict.py`
`code/tests/`, `code/docs/`	Unit tests; docs, including the architecture details and every decision with its reason
`code/artifacts/`, `code/report/`	Trained model and its model card; the sources of this report
`code/app/`	Optional: an interactive fault simulator to play with the model (Appendix F)

1. Data exploration and data quality

1.1 My approach

I worked in four steps, one notebook each (`code/notebooks/01` to `04`), and turned every fix into a tested function in `src/ahu_fdd/` so the model and production use exactly the same cleaning:

1. **What is in the data** (`01_data_overview`): the four files, coverage per machine, real against declared schedules, the labels.
2. **Can I trust it** (`02_data_quality`): every problem I found, how I proved it and how I fixed it. `clean_sensors()` runs all fixes in order, on train and test.
3. **What a healthy machine looks like** (`03_patterns`): only rows with the fan on and no fault label, because a fault is a change from normal.

4. **How each fault shows** (`04_faults`): the physics I expect, checked against the 28 labelled events, compared with the healthy rows of the same machine.

The data covers 40 AHUs in three buildings: an office (B1, 16 units), a hospital (B2, 12 units, 24/7, with CO2 and airflow) and a mall (B3, 12 units, no extra sensors). There are 2.05 million rows in train and 0.71 million in test. Only rows with the fan on are scored, and in those, faults are 2.4% of the rows: 28 events in 21 machines, 6 to 8 per fault type, lasting 2 to 15 days. **Assumption:** the data description says UTC+8 but not where. The office closed on exactly the seven weekdays that are Singapore 2025 public holidays, so I assume Singapore.

1.2 Data quality

Each building had its own problem, the same in all its machines, which points to how each BMS stored or exported the data, not to broken sensors. The test period is clean apart from one frozen sensor.

Problem	How I found it	What I did	What I would ask the operator
Office week stamped 8 hours early (B1, 1 to 8 June): duplicates on 1 June, a hole on 8 June, the office "running at night"	The three buildings share the same weather. The office outside temperature matches the mall exactly 8 hours later (0.4 °C gap) and not at the same time (3.6 °C); trying every offset from -12 to +12 h, the best is +8	Moved the block 8 hours later (`fix_clock_shift()`); 13 rows with an unknown clock dropped	Did the BMS change its time settings in early June?
One mall day saved twice (B3, 14 February)	3,439 duplicate pairs; 97.9% identical, the rest differ only by an empty value	Kept one row and filled empty values from the other copy (`merge_duplicates()`)	Was the data joined from overlapping exports?
Hospital outage (B2, 10 to 18 March, 200 hours, all 12 units at once)	Joined missing slots into blocks: one long block, all other gaps are 5-10 minutes	Left as a gap; no label falls in it	Was the BMS or its network down?
Fan power in watts (B3-AHU-04, 1 to 21 April)	Power 1,000 times the rated power of the fan, with the fan on and off	Divided by 1,000 (`fix_power_units()`)	Was the meter changed in April?
CO2 glitches (B2-AHU-03, January and February)	781 readings at exactly 0 or 5,000 ppm, the limits of the sensor	Set to empty (`blank_co2_limits()`)	Was this sensor checked?
Frozen sensors: return air of B1-AHU-07 (22 days, train), supply air of B2-AHU-08 (9 days, test)	Longest run of exactly the same value: normal sensors never stay flat over 1.5 hours	Readings flat for 6 hours or more set to empty (`blank_frozen()`)	Why did these sensors stop updating?
About 0.2% empty values in five sensors	Share of empty values per sensor and building	Kept empty	

The frozen supply air sensor in **test** matters most: it reads 13.1 °C, 0.1 °C above its setpoint, so the machine looks perfect. It is not one of the four faults, so the model must not raise an alarm there.

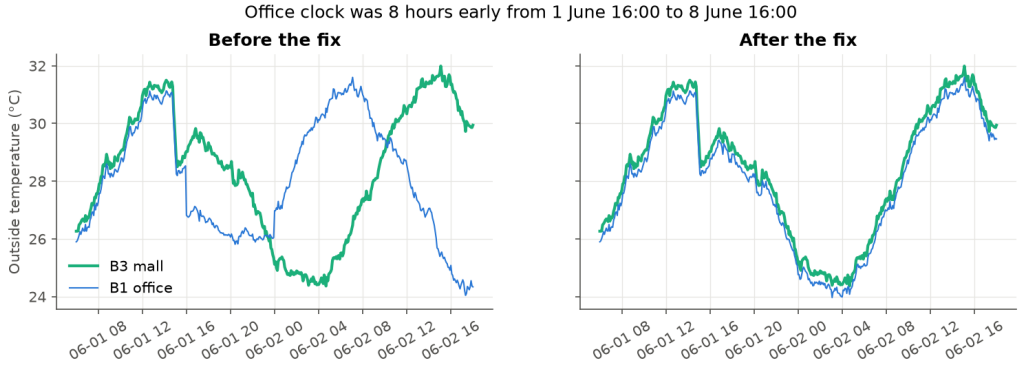


Figure 1: The office clock was 8 hours early for one week: the outside temperature of the office only matches the mall after the fix.

1.3 Patterns across buildings and units

- **A normal day.** The valve and the fan follow the outside temperature and the people inside: the valve is about 30% at night and 60% in the afternoon, with a dip at lunch in the office. The supply air stays near its setpoint.
- **Setpoints differ between machines** (12.5 to 14.5 °C), but each machine has a fixed one. So I always use the error `sat_c - sat_sp_c`, never `sat_c` alone.
- **Start-up looks like a fault.** In the first 15 minutes after the fan starts, the supply air is about 5 °C above its setpoint; after 30 minutes it is on target. I don't raise alarms in the first 30 minutes.
- **Afternoon storms,** about two per week: outside air drops 3 °C in minutes and the valves close a little. Smoothing over hours handles them.
- **Three machines are always at their limit** (B1-AHU-05 lobby, B3-AHU-02 cinema foyer, B3-AHU-09 retail): valve at 100% in 60-80% of their healthy rows, every month, in train and test, with the supply air about 2 °C too warm. They are labelled normal. A global threshold would raise an alarm on them every day, so **every feature compares a machine with its own normal**.
- **Normal drifts:** filters get dirty for months and are then replaced (B1-AHU-05: from 245 to 95 Pa on 1 July). I validate in time order and don't use absolute filter or fan power values.
- **Test looks like train:** Singapore has almost no seasons; July and August are 0.3 °C warmer.

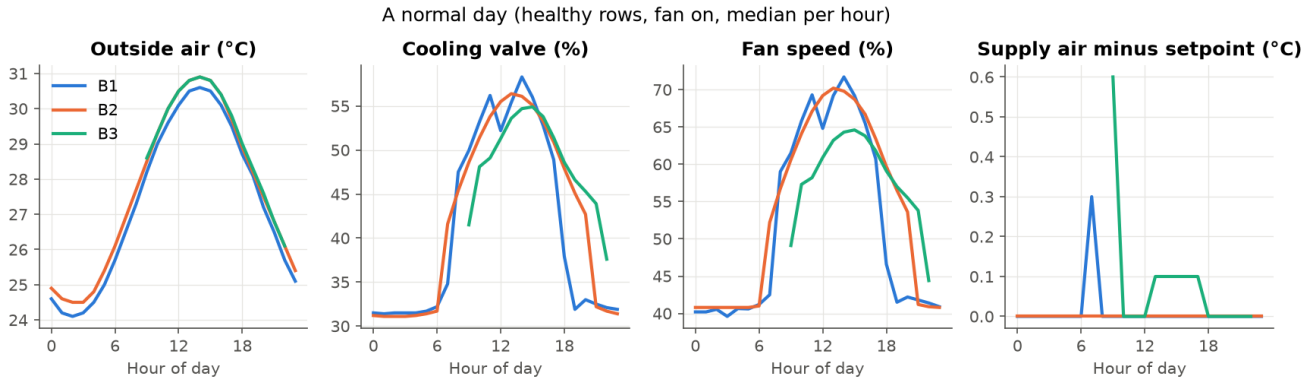


Figure 2: A normal day per building (healthy rows, fan on, median per hour): valve and fan follow the outside temperature, the supply air stays on target except at start-up.

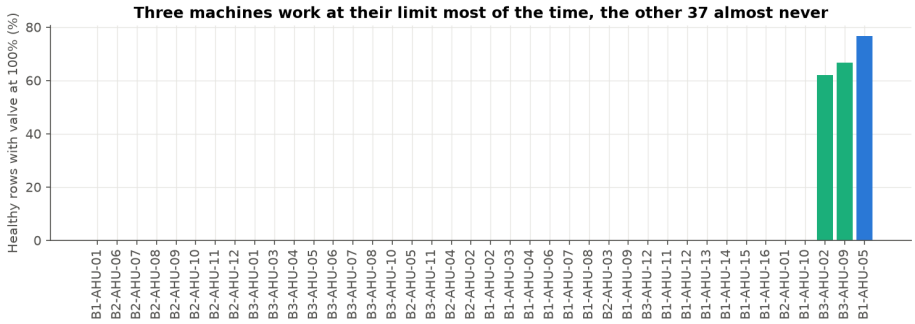


Figure 3: Share of healthy rows with the valve at 100%: three machines work at their limit most of the time, the other 37 almost never.

1.4 The four faults: physics and labels

I compared each labelled event with the healthy rows of the same machine. Every fault leaves a different fingerprint, which is what a model needs to tell them apart. (In the notebooks and figures I also use plain names, easier to explain to non-technical people: *stuck tap*, *stuck flap*, *weak radiator* and *lying thermometer*.)

Fault	What should change, and why	What the labelled data shows
Stuck chilled water valve (`chw_valve_stuck`)	The valve no longer follows its command, so the supply air goes off target and the controller pushes the command to a limit	Supply air 4 to 12 °C too warm (stuck nearly closed) or about 5 °C too cold (stuck open); command at 0% or 100% in 99% of the rows
Stuck outdoor air damper (`oa_damper_stuck`)	No outdoor air: mixed air equals return air. By the mixing formula, the outdoor air share x = (MAT - RAT) / (OAT - RAT) drops to 0, and CO2 rises	Outdoor air share from 0.10-0.20 to almost 0; flap command +40 points (the controller tries to compensate); CO2 +440 to +1,030 ppm where there is a sensor
Low coil capacity (`low_coil_capacity`)	The coil cools less for the same valve opening, so the controller opens the valve more	Valve about 20 points above what the same machine needs for the same cooling work; supply air stays on target until the valve reaches 100%
Supply air sensor bias (`sat_sensor_bias`)	The controller trusts the sensor, so the reading stays on target while the real air is too warm or too cold. Nothing shows while the fan runs	With the fan off for an hour, supply and mixed air should read the same: normally -0.5 °C, during the six events +2.0 to +2.4 °C or -2.5 to -3.2 °C

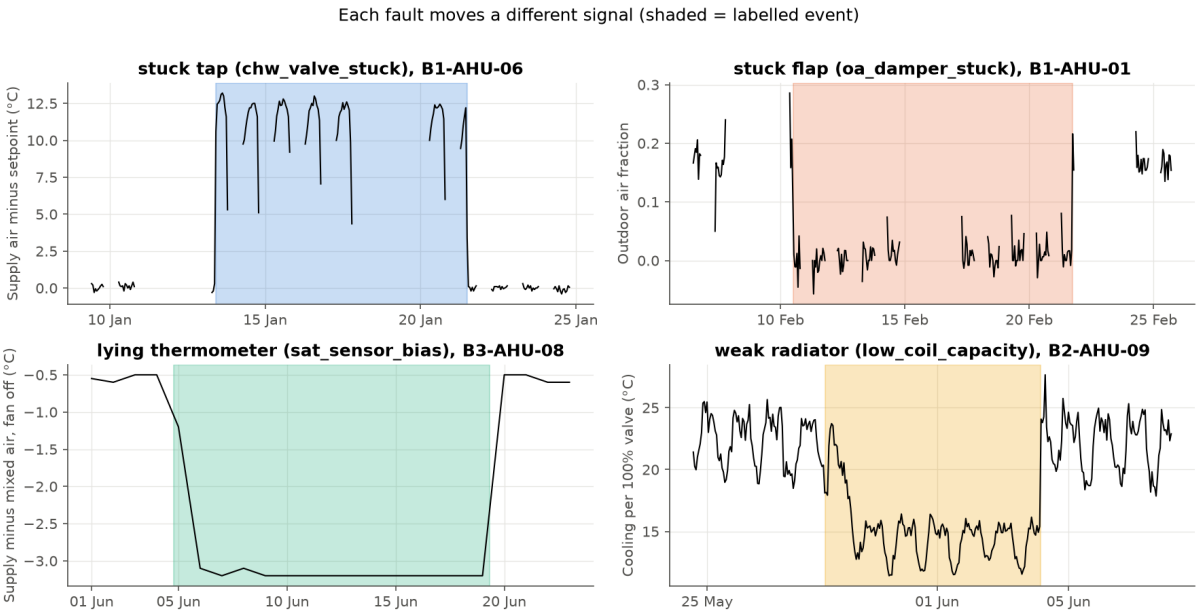


Figure 4: One real event of each fault, with the signal that shows it best (shaded: labelled event).

Do the labels agree? Yes. For the stuck valve and damper, the fingerprint starts and ends within an hour of the work order dates; the bigger gaps at the end of office events are weekends (the fan is off from Friday to Monday). The sensor bias is seen 11 to 41 hours late, because the check needs the fan off at night.

Looks like a fault but is not labelled:

- **B1-AHU-11, 22 to 30 April:** a textbook stuck damper (outdoor air share near 0, flap command 70% instead of 15%, CO2 from 980 to 1,700 ppm), and on 30 April at about 16:00 the data shows the repair: outdoor air comes back and CO2 drops in half an hour. There is no work order. I added it as a training label, because leaving it as normal would teach the model that a stuck damper is healthy.
- The three machines always at their limit look like a permanent low coil capacity, but I treat them as normal (undersized units), as the labels say.

Labelled but hard to see:

- The sensor bias on machines that never switch off (the hospital and two office units): the check at rest is impossible there.
- A stuck valve or low coil capacity on the three machines at their limit: their valve is already at 100%, so the fault hides in their daily struggle (B3-AHU-09's stuck valve is hard to separate from its normal).

2. Model

Full work in `code/notebooks/05_modelling.ipynb`; `python -m ahu_fdd.predict` rebuilds `predictions.csv` from the raw data and saves the model with its model card.

Framing. A combination: a **LightGBM classifier decides if** a machine has a fault, and the **physics rules of Section 1.4 decide which** one inside each alarm. The model gives the fewest false alarms; the rules get the type right more often. Every feature is a residual against **the same machine's normal**, built from the physics of Section 1.4: supply air error, the valve against the valve this machine needs for the same cooling work (fan speed times °C cooled), the outdoor air share from the mixing formula, CO2, and the check with the fan off (9 features, Appendix B). LightGBM because the problem is tabular and threshold-like, it handles missing sensors natively, and I keep it small (only 29 events to learn from). Class weights instead of resampling; no synthetic fault rows.

Validation. Across time: three blocks of about two months, cut on days with no event in progress; each block is predicted by a model trained on the other two, and each machine's normal comes only from the training blocks. **Across units:** a whole building hidden from training (see below). I score by event: found, right type, delay, false alarms per AHU per month (row accuracy is useless: always "normal" is right on 97.6% of rows). Rows become alarms: model at least 80% sure, alarms shorter than 2 hours dropped, pieces of one fault joined, one type per alarm.

Option (29 events, 240 machine-months)	Found	Right type	Median delay	False alarms per AHU-month
Physics rules	100%	93%	11 h	0.18
Logistic regression	76%	59%	4 h	1.48
Random forest	93%	79%	11 h	0.03
LightGBM	100%	86%	10 h	0.05
LightGBM + rule types (final)	**100%**	**93%**	**10 h**	**0.05**

0.05 false alarms per AHU per month is about one per machine every 20 months. The tests behind the settings (relative features only, 80% confidence, joining alarms) are in Appendix B.

Where it fails. Two low coil capacity events on machines that are always at their limit are called a stuck valve: their valve is already at 100%, so a weak coil can't push it further. A valve position sensor or a chilled water flow meter would separate them. The remaining false alarms are short (2 to 5 hours) and all in the mall. The sensor bias is found the first night at the earliest, and can't be seen at all on 24/7 machines.

A new AHU, and a building with fewer sensors. I hid a whole building from training and gave it only two weeks of unlabelled data to learn each machine's normal. The model found 100% of the faults in the office but only 50-60% in the hospital and the mall (the mall has no CO2, and most sensor bias examples are there). The rules found all of them. So a new building starts with **two weeks to learn its normal, then the rules only**, and moves to the model once it has its own confirmed work orders. With fewer sensors, the missing features are simply empty and the rules don't use them. Ideas to shorten this cold start, such as injecting simulated faults into a new building's healthy data, are in Appendix E.

Test period. 14 alarms in 40 machines over July and August. They include the eight candidate faults I could see by hand in Section 1 (same type and dates), and no alarm on the machine with the frozen supply air sensor.

3. How I would run this in production

The design target is to run the model of Section 2 for 25 buildings (about 600 AHUs) in the first year, and to send the technicians work orders they can trust. I chose **Azure with Databricks**: the buildings already run Windows and SQL Server, and it is the stack I used in production for a similar ML system (Delta tables, scheduled jobs, MLflow with a champion model). The lighter option, containers with Parquet files and a managed MLflow, would also work and cost less; I would pick it if the operator has no Azure. More detail (technology choices, alert rules, monitoring thresholds, onboarding plan) in `code/docs/05 - Architecture.md`.

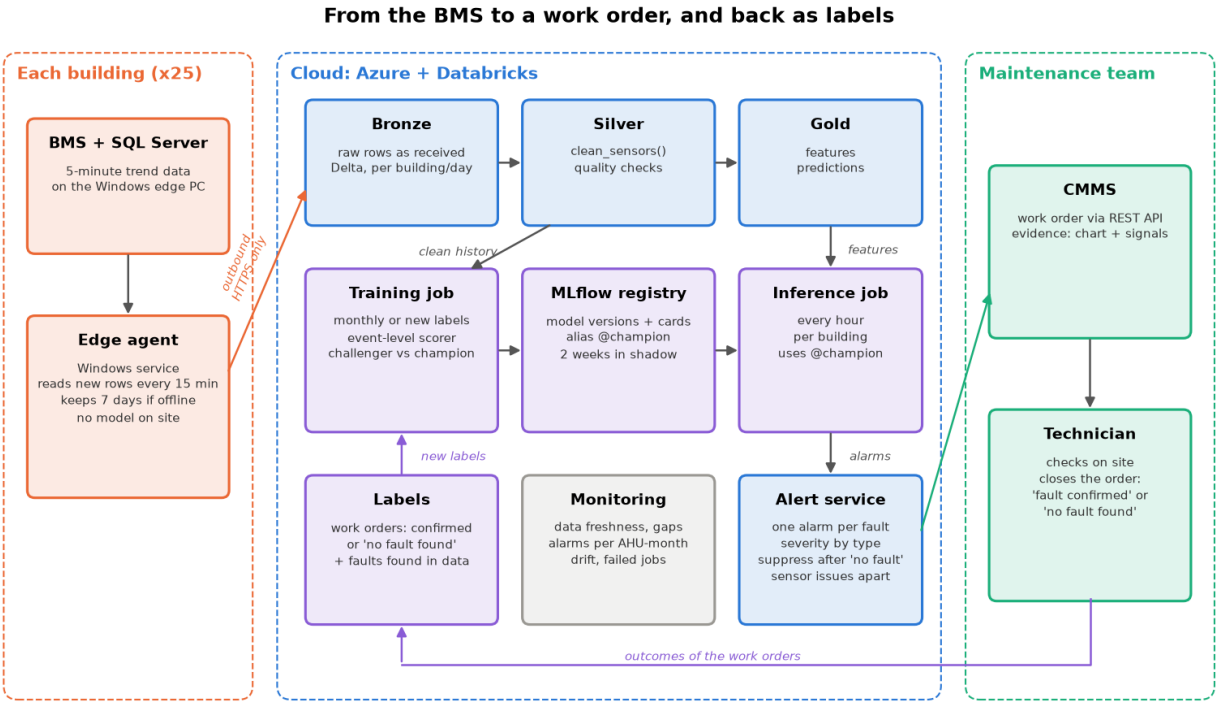


Figure 5: From the BMS to a work order, and back as labels.

Data ingestion and storage

- **At the edge:** a small Python agent runs as a Windows service on each edge PC. Every 15 minutes it reads the new rows from SQL Server (by last timestamp) and sends them over **outbound HTTPS** to an ingestion endpoint, with a token per site. If the connection fails, it keeps up to 7 days on disk and sends them later. Nothing listens on site, so no inbound connection is needed. The model does **not** run at the edge: one place to update it, and the edge PC is the least reliable part.
- **Storage:** three layers in Delta tables, partitioned by building and day. **Bronze** keeps the rows exactly as received, so I can always rebuild. **Silver** is the cleaned data (`clean_sensors()`). **Gold** has the features and the predictions.
- **Data quality:** the problems I found in Section 1 become automatic checks in the step to silver: clock shift (outside temperature compared with a neighbour building), duplicate rows, outages, wrong units, sensor limits and frozen sensors. A failed check fixes what it can and logs the rest. A frozen sensor raises a **"sensor issue"** ticket, kept apart from fault alarms, because it needs a different fix. Each building's batch gets a result: **pass, warn or block**. If it blocks (for example the clock is shifted or half the rows are missing), the alerts of that building pause until the data is fixed, so bad data never becomes a false alarm.

ML lifecycle

- **Training** runs once a month, or earlier when new confirmed work orders arrive. It reuses the code of Section 2: time-block validation and the event-level scorer.
- **Versioning:** every run is logged in MLflow (data period, code commit, parameters, scores) and registers a new model version with its model card, like the `artifacts/` folder of this repository.
- **Champion and challenger:** a new version (challenger) replaces the current one (champion) only if it passes **two gates**: a minimum level (for example, at least 90% of faults found and at most 0.1 false alarms per AHU per month) **and** a better score than the champion on the same validation data. Then it runs **two weeks in shadow mode**, predicting without sending alerts, and its alarms are compared with what the technicians

find. Rolling back means moving the `champion` alias back to the old version.
- **Environments:** code goes through the same branch flow as this repository (`develop`, `qa`, `main`), deployed with Databricks Asset Bundles to a dev, a QA and a production workspace. Each environment trains its own model on its own data; models are never copied between environments.
- **Labels from the field:** every day the system reads the closed work orders from the CMMS API. "Fault confirmed" becomes a label for that fault and period; "no fault found" becomes a normal label and a measured false alarm. This is the most valuable data: today there are only 29 events.

Inference strategy

Every hour, per building. Faults last days, an alarm needs at least 2 hours of evidence, and a supply air sensor bias can only be checked at night with the fan off. Every 5 minutes would cost more and gain minutes that don't matter for a visit the next day. Each run uses the last 48 hours of data, so the rolling features are complete.

Alerting and integration

The alert service turns predictions into work orders, and its job is to protect the technicians' trust:

- **One alarm per fault:** the pieces of the same fault are joined (the morning start-up doesn't split it) and an open alarm is updated, not repeated. It **closes** by itself after 24 hours of running back to normal, with a note on the work order, so old alarms don't stay open.
- **Severity by type and building:** a stuck valve or low coil capacity wastes energy and comfort now (high); a sensor bias or a stuck damper can usually wait a few days (medium), except a stuck damper in the hospital, where fresh air matters for patients (high).
- **Suppression:** after "no fault found", the same fault on the same AHU is silenced for 14 days, unless the signal gets clearly worse.
- **Work order:** created through the CMMS REST API, with the evidence: a small chart of the signals that triggered it and the plain fault name, so the technician knows what to check first.
- **Feedback:** the outcome of each order flows back as a label (see above).

Monitoring

- **Data:** freshness per building (an agent that stopped sending), share of missing rows, failed quality checks, frozen sensors.
- **Model:** alarms per AHU per month against the expected rate (0.05 in validation), share of alarms confirmed by technicians, and drift of the main features (for example, filters slowly change the fan signals).
- **Jobs:** failed or late runs.

All of it is in one dashboard (Databricks SQL) that reads from MLflow and the alert tables: champion and challenger versions with their scores, alarms by type and building, confirmed share, drift and data checks.

First month after go-live: the model runs in shadow for the first two weeks, then sends alerts with a weekly review with the facilities manager. If a site raises too many alarms, I raise its confidence threshold or minimum alarm length for that site before going further.

Scale and onboarding

- **Scale:** 2,000 AHUs are about 600,000 rows a day, under 100 MB. Jobs run per building in parallel: growing means more small jobs, not a bigger system.
- **A new building in under a week:**
  - Day 1: install the agent and its token.
  - Day 2: map the BMS point names to the standard names (one template per BMS vendor) and fill the AHU metadata (size, sensors).
  - Day 3: load the history already stored in SQL Server, and run the quality checks.
  - Days 4-5: if there are at least two weeks of history, learn each AHU's normal and start the **physics rules** in shadow mode. Otherwise the rules wait until two weeks of data arrive.
- **Why rules first:** in Section 2, a model trained on other buildings found only 50-60% of the faults in a building it had never seen, while the rules found all of them. The model takes over once the building has its own confirmed work orders.

4. Communicating results: an example

This section is an example scenario, to show how I explain a model to people who depend on it. Two weeks after a pilot starts in one building, the facilities manager writes: the system raised 9 alerts last week, the technicians checked 4 and found nothing wrong, they are starting to ignore the alerts, and the owner wants two more buildings next month.

The reasons behind both texts are in Appendix A.

4.1 Reply to the facilities manager

Subject: Pilot alerts: review and next steps

Hi,

Thank you for the clear feedback, and for having the alerts checked. I understand the concern: if your team stops trusting the alerts, the system is worth nothing, so fixing this is my first priority.

Why this can happen: the system learns what is normal for each machine, and your building is new to it. In the first weeks it can take unusual but healthy behaviour for a fault, for example a unit that is a bit small for its area and always works at full power. Some faults are also hard to see on a visit: a temperature sensor that reads 2 degrees too high looks fine unless you check it with a second thermometer. So, during the next weeks, I will send a short note with each alert saying what to check on site, for example measuring the air temperature with a hand thermometer.

What we change this week:

- I will go through the 9 alerts one by one with the data, and send you a short answer for each by Wednesday: real problem, healthy machine, or our error.
- From now on, your technicians only get the alerts the system is very sure about. We check the others first.
- For the doubtful cases, I would like to visit the machine together with one of your technicians.

What you could tell the owner: the pilot is being tuned to this building before it grows. We suggest connecting the two new buildings next month as planned, but without alerts: the system learns them quietly. Alerts start there only when your building shows that most alerts are real. Our target is at least 7 out of 10 checked alerts confirmed, two weeks in a row. The owner keeps the timeline, and your team doesn't get the same problem in three buildings.

Two small things would help me a lot: the notes from the 4 visits, and for each new alert, a quick "fault found" or "nothing found" in the work order system.

Could we have a 30-minute call on Monday?

Best regards,
Sergio

4.2 Internal note to the project lead

Pilot building: the alerts are losing the technicians' trust

What I think is happening. 9 alerts in a week is about ten times what validation suggests for a building of this size, so I suspect our setup more than the idea of the model. In order of likelihood:

1. **We turned alerts on too early.** In my tests, a model trained on other buildings found only about half of the faults in a new one; our own plan says rules first and two weeks in shadow mode.
2. **Healthy machines that look faulty.** In the training data, 3 of 40 units always run at full valve and look like a weak coil. If this building has some, they alarm every week.
3. **A data problem** (clock shift, wrong units, frozen sensor) that the quality gate didn't stop.
4. **Real faults that are hard to confirm on site,** such as a biased sensor, with alerts that don't say what to check.

Also, 5 of the 9 alerts were never checked, so we don't know the real hit rate yet.

Next two weeks.

- Days 1-2: review the 9 alerts with the raw data and label each one: real fault, healthy but unusual, data problem or unclear.
- Week 1: shadow mode again for the unclear fault types, a higher threshold for this site, units at their limit marked as normal, a manual "what to check" note with each alert, and one or two joint visits.
- Week 2: only high-confidence alerts, a weekly 30-minute review with the facilities manager, and tracking of

the confirmed share.

Scope question for you. The manual “what to check” notes are part of this review. Making it a product feature (each alert explains its cause and recommends an action) may be outside the current scope. If it is, I would propose it as a change request: it is the change that would help the technicians most.

What I need from the client. The notes of the 4 checked alerts and the outcome of every new one; one technician for one or two joint visits; the BMS history and any setting or maintenance change during the pilot; sensor calibration dates; and which units they already know are undersized.

Recommendation on the rollout. Don’t turn alerts on in the two new buildings next month. Do connect them next month in shadow mode: the system learns each machine’s normal without sending anything. Turn their alerts on only when the pilot reaches a clear target: at least 7 out of 10 checked alerts confirmed, two weeks in a row. If the pilot doesn’t get there within four weeks, we stop and review the approach with the client. The owner keeps the start date, and we don’t repeat the problem in three buildings.

Appendix

Details behind the main pages. Not needed to follow the report.

A. Why I answered this way in the example

The reply to the facilities manager. They are not technical, they are frustrated, and they have to tell the owner something. So I followed five rules:

1. **Acknowledge first, defend later.** Their point is correct: alerts that technicians ignore are worthless. Saying so in the first lines, in a professional tone, keeps the conversation open.
2. **No blame, and no guilt I can't prove yet.** "Nothing found" doesn't always mean "nothing broken": some faults can't be seen without the right test (Section 1: a supply air sensor bias is invisible while the machine runs). Instead of saying whose fault it was, the reply turns this into an action: during the review I send a "what to check" note with each alert. I keep it manual on purpose: turning it into a product feature may be outside the agreed scope, so it goes to the project lead as a possible change request, not as a promise to the operator.
3. **Plain words and one example per idea.** No model, threshold or feature. "A unit that is a bit small for its area" is the healthy machine at its limit of Section 1; "a sensor that reads 2 degrees too high" is the sensor bias.
4. **Concrete actions with a date,** small enough to do this week and inside the current scope: review of the 9 alerts by Wednesday, only high-confidence alerts, a "what to check" note with each alert, a joint visit.
5. **Give them a message for the owner** that keeps the owner's timeline but protects the technicians: connect the new buildings next month without alerts, and turn alerts on when a clear target is met. A yes/no answer would put the facilities manager in a bad position.

The internal note. Here I can be direct and use numbers. Every hypothesis comes from what I saw in the data:

Hypothesis in the note	Where it comes from
Alerts turned on too early in a new building	Section 2: a model trained on other buildings found about half of the faults of a building it never saw; the rules found all of them
Healthy machines that look faulty	Section 1: 3 of 40 units always run at full valve and look like a weak coil, labelled normal in the data
A data problem	Section 1: each building had its own problem (clock shift, repeated day, outage, wrong units, frozen sensors)
Real faults hard to confirm on site	Section 1: the sensor bias only shows with the fan off
9 alerts is about ten times the expected rate	Section 2: 0.05 false alarms per AHU per month in validation, so less than one a month for 12-16 units

The rollout recommendation. I didn't say "go" or "stop". Connecting the new buildings in shadow mode costs nothing in trust and gains time, because each machine needs about two weeks of data to learn its normal anyway (Section 3, onboarding). The target (7 of 10 checked alerts confirmed, two weeks in a row) is simple enough for the owner to follow, and the four-week limit makes sure we don't hide the problem if it doesn't improve.

B. Model details

The 9 features. Each one is compared with the same machine's normal, learned from its healthy rows.

Feature	What it measures	Mainly for
`sat_error_dev`	Supply air minus setpoint, last hour, vs this machine's usual error	Stuck valve
`valve_dev`	Valve command, last hour, vs this machine's usual opening	Stuck valve
`valve_at_limit_1h`	Share of the last hour with the valve command at 0% or 100%	Stuck valve
`valve_excess`	Valve above what this machine needs for the same cooling work (fan speed × °C cooled), last 3 hours	Low coil capacity

Feature	What it measures	Mainly for
`oa_fraction_1h`	Outdoor air share from the mixing formula, last hour	Stuck damper
`oa_fraction_dev`	The same share vs this machine's normal	Stuck damper
`damper_dev`	Damper command above its usual value (the controller compensates)	Stuck damper
`co2_dev`	CO2 above this machine's normal (office and hospital only)	Stuck damper
`rest_offset_dev`	Last check with the fan off for an hour: supply minus mixed air, vs normal	Sensor bias

The tests behind the final settings (same validation and scorer as the main table):

Test	Option	Alarms raised	Found	Right type	False alarms per AHU-month
A. Features (model alone)	Raw + relative signals	50	90%	69%	0.04
	Relative signals only	52	100%	86%	0.05
B. Confidence	50%	128	100%	93%	0.36
	70%	67	100%	93%	0.11
	80%	52	100%	93%	0.05
	90%	38	93%	86%	0.01
C. Join pieces of a fault	Not joined	193	100%	93%	0.05
	Under 2 hours apart	70	100%	93%	0.05
	Under 6 hours apart	52	100%	93%	0.05

- **A:** with raw values, the three machines always at their limit look like a stuck valve, so the model misses faults and mixes types.
- **B:** 80% is the highest confidence that still finds every fault.
- **C:** joining the pieces of one fault (the morning start-up cut them) gives fewer alarms for the same faults; it never adds false alarms.

Right type per fault (final model): stuck valve 100%, stuck damper 100%, sensor bias 100%, low coil capacity 71% (the two errors are the machines at their limit).

C. Alarms in the test period

The 14 alarms in `predictions.csv` (July and August), from the first to the last hour of each:

AHU	Fault	From	To	Days
B3-AHU-11	`sat_sensor_bias`	Wed 02 Jul 13:05	Wed 02 Jul 16:50	0.2
B1-AHU-01	`chw_valve_stuck`	Mon 07 Jul 15:15	Fri 11 Jul 19:25	4.2
B1-AHU-03	`low_coil_capacity`	Wed 09 Jul 14:15	Fri 11 Jul 19:25	2.2
B3-AHU-05	`sat_sensor_bias`	Sat 12 Jul 10:30	Sat 12 Jul 12:35	0.1
B2-AHU-12	`oa_damper_stuck`	Sat 12 Jul 16:40	Thu 24 Jul 15:25	11.9
B1-AHU-09	`low_coil_capacity`	Tue 15 Jul 09:05	Fri 18 Jul 11:25	3.1
B1-AHU-12	`sat_sensor_bias`	Tue 22 Jul 08:05	Wed 06 Aug 19:25	15.5
B3-AHU-05	`chw_valve_stuck`	Tue 22 Jul 09:45	Tue 29 Jul 22:55	7.5
B1-AHU-16	`chw_valve_stuck`	Wed 23 Jul 09:30	Mon 28 Jul 17:30	5.3
B3-AHU-06	`sat_sensor_bias`	Fri 25 Jul 09:30	Mon 11 Aug 20:40	17.5
B3-AHU-03	`low_coil_capacity`	Sat 26 Jul 15:25	Sat 26 Jul 19:00	0.1
B3-AHU-03	`low_coil_capacity`	Sun 27 Jul 11:25	Mon 28 Jul 17:55	1.3

AHU	Fault	From	To	Days
B2-AHU-07	`oa_damper_stuck`	Wed 30 Jul 00:10	Tue 05 Aug 06:50	6.3
B3-AHU-01	`oa_damper_stuck`	Thu 21 Aug 09:45	Tue 26 Aug 20:50	5.5

Eight of them (stuck valves on B1-AHU-01, B1-AHU-16 and B3-AHU-05; stuck dampers on B2-AHU-07, B2-AHU-12 and B3-AHU-01; sensor bias on B1-AHU-12 and B3-AHU-06) match the fingerprints I could see by hand in the data before training. The short ones (under a day) look like the false alarms of validation. B2-AHU-08, whose supply air sensor is frozen for 9 days, has no alarm.

D. Questions for the building operator

All in one list, from the data work and the design (also in `code/docs/decisions.md`):

1. Did the office BMS change its time settings in early June? Was the mall data joined from overlapping exports? Was the hospital BMS down from 10 to 18 March?
2. Was a meter changed on B3-AHU-04 in April? Why did the return air sensor of B1-AHU-07 and the supply air sensor of B2-AHU-08 stop updating?
3. Was there a work order for B1-AHU-11 at the end of April (stuck damper in the data, none in the labels)?
4. Are B1-AHU-05, B3-AHU-02 and B3-AHU-09 known to be undersized?
5. Where are the buildings, and which holiday calendar do they follow (I assumed Singapore)?
6. Can technicians close each alert with “fault confirmed” or “no fault found”?
7. Can each edge PC reach one HTTPS endpoint, and how much trend history does each BMS keep?

E. If I had more time

In order of what I think would help an operator most:

1. **Cold start: a new building with no labels.** Today a new building runs on the physics rules until it has its own confirmed work orders, because the model trained on other buildings found only 50-60% of the faults in a building it never saw. To shorten that time:
   - **Inject faults into healthy data.** Take a healthy machine of the new building and simulate each fault in its own data (hold the valve, close the damper, add 2 °C to the supply sensor, cut the coil capacity), with a known start and size. The model gets labelled examples from day one, and I can measure the detection delay on each building before going live. A first version of this is the simulator of Appendix F.
   - **Train on more general signals.** Most features are already relative to each machine; adding the building type and the sensors fitted as features, and training across buildings with more of them, should make the model transfer better.
   - **Ask for the right labels first.** Send the technicians first the alerts where the model is least sure, so each visit teaches the model as much as possible (active learning).
2. **Sensor bias on 24/7 machines.** The check at rest needs the fan off, so it can’t run on the hospital. Options: a short planned stop at night once a week, or a “virtual sensor” that estimates the supply air from the chilled water temperatures and the valve, and compares it with the real sensor.
3. **Explain each alert and recommend an action.** For every alarm: the two or three signals behind it (feature contributions) and a plain “what to check on site” line per fault type. This is the change request of Section 4: it would help the technicians most.
4. **Severity by cost.** Estimate the energy wasted by each fault (cooling lost or extra fan and valve work, in kW) and rank alerts by money and comfort, not only by fault type.
5. **Separate the hard pair.** A stuck valve and a low coil capacity look alike on the machines at their limit. A valve position feedback signal or a chilled water flow meter on those three units would remove most of the type errors.
6. **More model options and tuning.** Compare XGBoost and CatBoost, tune the parameters inside the time blocks, try a two-step model (fault yes or no, then which one), and calibrate the probabilities so “80% sure” really means 80%.
7. **Use the unlabelled data.** Self-training on very confident alarms, and an automatic check of every work order against its fingerprint, to find missing labels like B1-AHU-11 and wrong dates.
8. **Production pieces of Section 3 in code.** MLflow with the champion and challenger rule (two gates), a Dockerfile, and the data quality gate (pass, warn, block) as a job.

F. Fault simulator

An interactive app (`code/app/`, Streamlit) to play with the model. Pick a healthy AHU, inject one of the four faults with the size you want, and see if and when an alarm is raised. It uses the same pipeline as `predictions.csv`.

Run it from `code/` (it needs the sample data, like the notebooks):

```
pip install -r app/requirements-app.txt
streamlit run app/streamlit_app.py
```

How to use it. In the sidebar:

1. **Machine and period:** any of the 40 AHUs and the first of the 14 days shown.
2. **Fault to inject:** the type, the day it starts, how long it lasts and its size (real valve position, outdoor air left, sensor bias in °C, coil capacity left). "No fault" shows the false alarms on healthy data.
3. **Alarm settings:** confidence needed, minimum alarm length and gap to join alarms (defaults: the settings of `predictions.csv`).

The page shows if the fault is found, the delay, the type and the false alarms (scored by event), and three charts: the signal that shows the fault (healthy and faulty), how sure the model is, and the alarms. Two things worth trying: lower the confidence with "No fault" to see false alarms appear, and shrink a fault to find the smallest one the model still catches.

How the faults are simulated (`src/ahu_fdd/simulate.py`, with unit tests): simple physics that reproduces the fingerprints of Section 1.4, not a full model of the unit. A stuck valve stays at the chosen opening, so the coil cools as if it were there and the controller pushes the command to a limit. A stuck damper lets in only part of the outdoor air, so mixed air moves to return air and CO2 rises. A sensor bias moves the reading with the fan off, while the controller hides it with the fan on. A weak coil makes the valve open more, and the air warms once the valve reaches 100%.

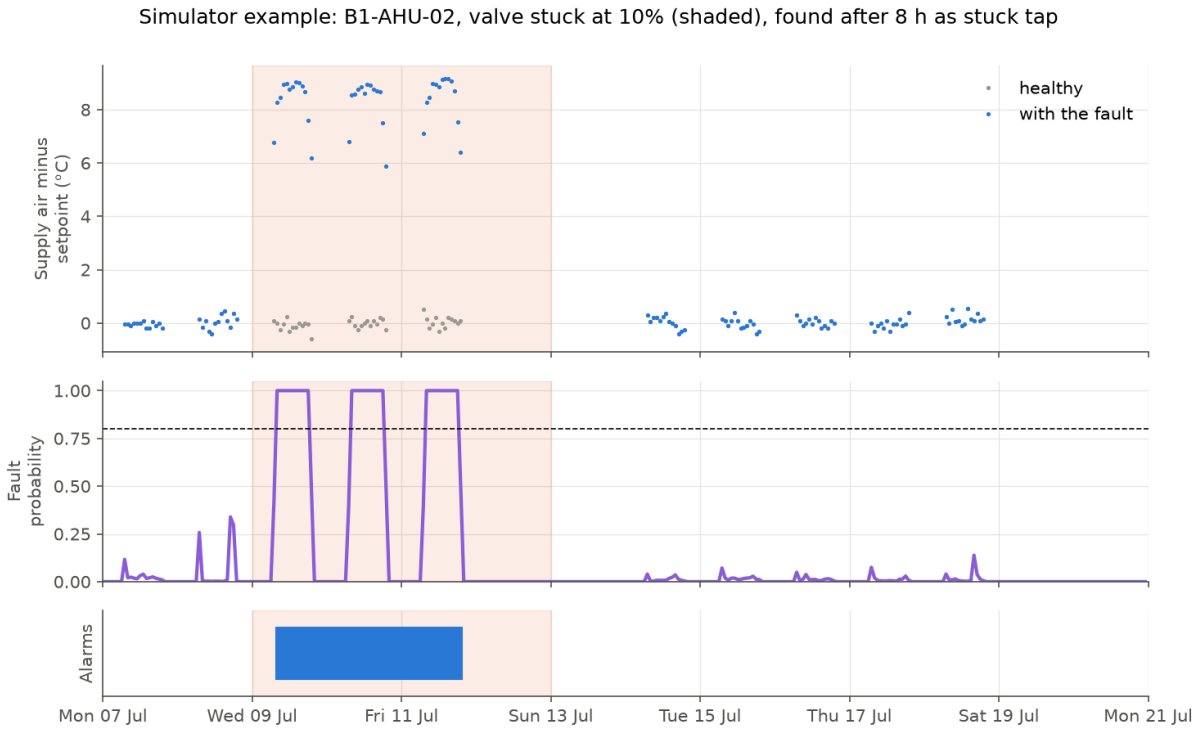


Figure 6: The simulator on B1-AHU-02 (office): the valve stuck at 10% for four days (shaded). The supply air goes 9 °C above its setpoint, the model is sure within the first morning, and one alarm of the right type is raised after 8 hours, with no false alarm.

G. How to run and check everything

From `code/`, after `pip install -r requirements.txt` and `pip install -e .` (details in `code/README.md`). About 12 to 15 minutes in total.

Step	Command	Time	What you should see
Unit tests	`pytest -q --cov`	~5 s	All pass, coverage about 91%
Lint	`ruff check .`	~2 s	All checks passed
Train and predict	`python -m ahu_fdd.predict`	~1 min	711,994 rows, 14 alarms, model card saved
Notebooks 01 to 04	Run all	~30-40 s each	No errors
Notebook 05	Run all	~6-8 min	No errors, same results as Section 2
This report	`python report/build.py`	~25 s	9 main pages + appendix
Fault simulator	`streamlit run app/streamlit_app.py`	~30 s	Opens in the browser (Appendix F)

There is no separate training command: `python -m ahu_fdd.predict` validates, trains the final model on January to June, predicts the test period and saves the model. Run it before notebook 05, whose last section reads `predictions.csv`.